Backend Intern Assignment: Ticket System in Golang

Two-Day Project Brief

Objective

Build a small backend service for a ticket system where a user can register, login, create tickets, view only their own tickets, and update the status of their own tickets.

What This Tests

- Problem-solving and API design
- Authentication and ownership-based authorization
- Golang code quality and basic LLD
- Dockerization and real deployment readiness

Scope

- No admin role is required.
- No ticket assignment flow is required.
- No comments module is required.
- No complex database schema is required. Use in-memory storage, SQLite, PostgreSQL, or any simple persistent store.

Implementation Contract

- Language: Golang.
- The service must expose REST APIs and run on port 8080.
- Authentication must use JWT. Protected APIs must require Authorization: Bearer <token>.
- Passwords must be stored as hashes, not plain text.
- A user must only be able to view and update tickets created by that same user.
- Supported ticket statuses: open, in_progress, closed.
- A closed ticket must not be reopened.
- Responses should be JSON and use meaningful HTTP status codes.
- The implementation will be checked using an automated hidden test suite, so endpoint names and basic field names must match the contract.

Required Endpoints

Method	Endpoint	Purpose
GET	/health	Health check
POST	/auth/register	Register user
POST	/auth/login	Login and return JWT
POST	/tickets	Create ticket
GET	/tickets	List logged-in user tickets
GET	/tickets/{id}	Get own ticket by ID
PATCH	/tickets/{id}/status	Update own ticket status

Required Status Flow

```
open -> in_progress -> closed
closed -> cannot move back to open or in_progress
```

Mandatory Deployment

- A working Dockerfile is mandatory.
- The application must be deployed on any free or no-cost hosting platform chosen by the candidate.
- The deployed service must expose the same APIs as the local Docker setup.
- The deployed /health endpoint must be publicly accessible.
- Submission without a deployed URL will be considered incomplete.
- No paid cloud service is required. The candidate can use any free-tier or no-cost platform that supports Go or Docker deployment.

Local Run Contract

```
docker build -t ticket-system .
docker run -p 8080:8080 ticket-system
curl http://localhost:8080/health
```

Expected Health Response

```
{
  "status": "ok"
}
```

Two-Day Timeline

Day	Expected Progress
Day 1	Project setup, auth, JWT middleware, ticket creation, ticket listing, own-ticket access.
Day 2	Status update, validations, Dockerfile, deployment, README cleanup, final submission.

Submission Requirements

- GitHub repository link.
- Deployed application URL.
- Public health check URL.
- README with local run command, Docker run command, deployment URL, and any assumptions.
- .env.example file, if environment variables are used.

Important Note

Keep the implementation simple. Do not over-engineer. Focus on correctness, clean ownership checks, readable Go code, and a working deployed service.